**UNIVERSIDADE FEDERAL DE PELOTAS**
**Centro de Desenvolvimento Tecnológico**
**Curso de Bacharelado em Ciência da Computação**




Trabalho de Conclusão de Curso

# Gerenciamento Multi-Estruturado de Dados Móveis Gamificados: Integrando Fontes Heterogêneas

**Rávilon Aguiar dos Santos**

Pelotas, 2026

**Rávilon Aguiar dos Santos**

# Gerenciamento Multi-Estruturado de Dados Móveis Gamificados: Integrando Fontes Heterogêneas

Trabalho de Conclusão de Curso apresentado ao Centro de Desenvolvimento Tecnológico da Universidade Federal de Pelotas, como requisito parcial à obtenção do título de Bacharel em Ciência da Computação.

Orientadora: Prof. Dr. Ana Marilza Pernas Fleischmann

Pelotas, 2026

# Resumo

SANTOS, Rávilon Aguiar dos. **Gerenciamento Multi-Estruturado de Dados Móveis Gamificados: Integrando Fontes Heterogêneas**. Orientadora: Ana Marilza Pernas Fleischmann. 2026. 36 f. Trabalho de Conclusão de Curso (Ciência da Computação) – Centro de Desenvolvimento Tecnológico, Universidade Federal de Pelotas, Pelotas, 2026.

Edificações inteligentes produzem dados contínuos e heterogêneos, sujeitos a duplicação, atraso e evolução de formato. Para apoiar análises e mecanismos de gamificação da eficiência energética, é necessário preservar temporariamente a evidência original, normalizar medições e manter rastreabilidade entre eventos e recompensas. Este trabalho apresenta o projeto e a implementação da arquitetura de dados da plataforma PROCEL, desenvolvida no contexto de uma iniciativa da Universidade Federal de Pelotas. No título, dados móveis designam os registros que pessoas, atuando como sensores, produzirão ao participar do sistema por meio da futura aplicação de gamificação; o recorte deste trabalho é o backend que receberá, integrará e relacionará esses registros à telemetria ambiental. A pesquisa seguiu a Ciência do Projeto, relacionando problema, objetivos, desenvolvimento, demonstração e avaliação do artefato. A solução adota persistência poliglota: eventos brutos recebidos por REST ou MQTT são armazenados no MongoDB, enquanto o domínio canônico, agregados e gamificação são persistidos no PostgreSQL. Um serviço de telemetria implementa retenção configurável, idempotência e reprocessamento; uma API executa canonicalização, agregação por janelas, avaliação de missões e concessão idempotente de experiência; e um console web apoia a operação. A avaliação foi delimitada a uma versão identificada do código e examinou cenários automatizados de duplicação, conflito, concorrência, recuperação e fronteiras temporais, além de um benchmark JMH. Um relatório anterior de testes de sanidade (*smoke tests*) foi usado apenas como evidência operacional complementar. A aplicação móvel não foi implementada nem avaliada neste TCC. Nos cenários inspecionados, restrições e testes exercitam as propriedades centrais, enquanto o benchmark favorece o motor simples nas regras instantâneas medidas. Não foram realizados ensaios com ocupantes, carga de produção ou falhas distribuídas de longa duração; portanto, não se afirma economia de energia nem garantia geral de desempenho ou disponibilidade.

Palavras-chave: Internet das Coisas; Telemetria; Persistência Poliglota; Gamificação; Eficiência Energética.

# Abstract

SANTOS, Rávilon Aguiar dos. **Multi-Structured Management of Gamified Mobile Data: Integrating Heterogeneous Sources**. Advisor: Ana Marilza Pernas Fleischmann. 2026. 36 f. Undergraduate Thesis (Computer Science) – Technology Development Center, Federal University of Pelotas, Pelotas, 2026.

Smart buildings produce heterogeneous data that may be duplicated, delayed, or changed in format. This work presents the design and implementation of the PROCEL data architecture, developed within an energy-efficiency initiative at the Federal University of Pelotas. In the title, mobile data denotes records that people, acting as sensors, will produce through the future gamification application; this work is limited to the backend that will receive, integrate, and relate those records to environmental telemetry. Following Design Science, the solution adopts polyglot persistence: raw REST or MQTT events are stored in MongoDB, whereas the canonical domain, analytical aggregates, and gamification data are stored in PostgreSQL. A telemetry service provides configurable retention, idempotency, and reprocessing; a domain API performs canonicalization, windowed aggregation, mission evaluation, and idempotent experience grants; and a web console supports operation. Evaluation was bounded to an identified code revision and examined automated scenarios for duplication, conflicts, concurrency, recovery, and temporal boundaries, as well as a JMH benchmark. An earlier smoke-test report was treated only as supplementary operational evidence. The mobile application was neither implemented nor evaluated in this thesis. The inspected constraints and tests exercise the core properties, and the benchmark favors the simple engine in the measured instantaneous-rule scenarios. No user study, production-load test, or long-running distributed-failure experiment was conducted; therefore, the work does not claim energy savings or general performance and availability guarantees.

Keywords: Internet of Things; Telemetry; Polyglot Persistence; Gamification; Energy Efficiency.

# Lista de Figuras


Figura 1 Arquitetura do PROCEL, objetivos dos componentes e relações . . 18
Figura 2 Console de administração: medições recebidas por MQTT e resultados de qualificação . . . . . 19
Figura 3 Console de administração: definição de grupos e regras de qualificação . . . . . 20
Figura 4 Console de administração: edição de missão e recompensa em XP 20
Figura 5 Relações entre identidade, autorização e gamificação . . . . . 22
Figura 6 DER do núcleo de sensores, regras e qualificação; todas as relações indicadas são 1:N . . . . . 23
Figura 7 Fluxo de canonicalização e qualificação de um valor . . . . . 25
Figura 8 Rastreabilidade do dado bruto ao efeito de gamificação . . . . . 26

# Lista de Tabelas


Tabela 1 Síntese comportamental e arquitetural dos trabalhos relacionados . 14

Tabela 2 Correspondência entre a DSR e as atividades executadas . . . . . 15

Tabela 3 Perfis e identidades técnicas do controle de acesso . . . . . 21
Tabela 4 Resumo das estruturas persistentes identificadas . . . . . 23
Tabela 5 Exemplos de parâmetros tipados no modelo canônico . . . . . 24
Tabela 6 Exemplo de regras para o parâmetro temperatura . . . . . 25

Tabela 7 Matriz de propriedades, cenários e alcance das evidências . . . . . 28

# Lista de Abreviaturas e Siglas

| | |
|---|---|
| **API** | Application Programming Interface |
| **DSR** | Design Science Research |
| **IoT** | Internet of Things |
| **JWT** | JSON Web Token |
| **MQTT** | Message Queuing Telemetry Transport |
| **QoS** | Quality of Service |
| **REST** | Representational State Transfer |
| **TTL** | Time to Live |
| **XP** | Experience Points |

# Sumário


**1 Introdução** . . . 9
**1.1 Contexto e motivação** . . . 9
**1.2 Problema e objetivos** . . . 10
**1.3 Contribuições e organização** . . . 10

**2 Fundamentação Teórica** . . . 11
**2.1 IoT, heterogeneidade e canonicalização** . . . 11
**2.2 Persistência poliglota** . . . 11
**2.3 Fluxos, idempotência e janelas** . . . 11
**2.4 Gamificação e energia** . . . 12
**2.5 Segurança e observabilidade** . . . 12

**3 Trabalhos Relacionados** . . . 13

**4 Metodologia** . . . 15
**4.1 Delimitação do artefato e protocolo de avaliação** . . . 16

**5 Arquitetura e Desenvolvimento** . . . 18
**5.1 Visão geral** . . . 18
**5.2 Console de administração** . . . 19
**5.3 Perfis de usuário e controle de acesso** . . . 21
**5.4 Modelagem relacional do domínio de sensores** . . . 22
5.4.1 Tipo, instância e localização do sensor . . . 23
5.4.2 Definição e valor de parâmetros . . . 24
**5.5 Modelagem das regras de qualificação** . . . 24
5.5.1 Exemplo completo de qualificação . . . 25
5.5.2 Relação entre qualificação e missões . . . 25
**5.6 Telemetria bruta e MQTT** . . . 26
**5.7 Domínio canônico e análise** . . . 27
**5.8 Missões e XP** . . . 27

**6 Avaliação e Discussão** . . . 28
**6.1 Critérios e resultados funcionais** . . . 28
**6.2 Relatório de testes de sanidade** . . . 29
**6.3 Experimento dos motores de regras** . . . 29
**6.4 Discussão e ameaças à validade** . . . 30

**7 Conclusão** . . . 31

**8 Artefatos técnicos consultados** . . . 32

**Referências** . . . 33

# 1 Introdução

## 1.1 Contexto e motivação

A Internet das Coisas (IoT) amplia a observação do mundo físico, mas exige arquiteturas capazes de transportar, armazenar e interpretar fluxos heterogêneos (Gubbi; Buyya; Marusic; Palaniswami, 2013; Al-fuqaha; Guizani; Mohammadi; Aledhari; Ayyash, 2015). Em edificações, leituras de energia, temperatura, umidade, qualidade do ar e presença diferem em protocolo, frequência, unidade e formato. Além do valor, identidade, instante, localização, origem e qualidade alteram a interpretação da observação (Sagl; Resch; Blaschke, 2015).

O projeto "Validação de sistema de gamificação, gestão e automação na nuvem para eficiência energética de edificações públicas com foco inicial em universidades", conduzido na UFPel, demanda uma base capaz de transformar telemetria em informação útil. Estudos indicam que medição real, feedback, metas e elementos lúdicos podem influenciar consumo (Orland; Ram; Lang; Houser; Kling; Coccia, 2014; Oppong-tawiah; Webster; Staples; Cameron; Guinea; Hung, 2020; Iria; Fonseca; Cassola; Barbosa; Soares; Coelho; Ozdemir, 2020); contudo, os efeitos dependem do contexto, dos usuários e do desenho da intervenção (Hamari; Koivisto; Sarsa, 2014; Morganti; Pallavicini; Cadel; Candelieri; Archetti; Mantovani, 2017; Beck; Chitalia; Rai, 2019).

O título sintetiza três dimensões desse contexto. "Multi-estruturado" refere-se à combinação de eventos documentais, entidades relacionais, agregados e estruturas de gamificação. "Fontes heterogêneas" abrange sensores e produtores que diferem em protocolo, formato e semântica. "Dados móveis gamificados" corresponde aos registros produzidos pela participação ativa das pessoas, tratadas como sensores contextuais, por meio da aplicação de gamificação que consumirá o backend (Sagl; Resch; Blaschke, 2015). Essa aplicação cliente integra o contexto do projeto, mas sua implementação e sua avaliação estão fora do recorte deste TCC.

Antes de avaliar o efeito comportamental, uma mensagem duplicada não pode gerar duas medições ou recompensas; falhas não devem eliminar a evidência original; e mudanças externas não devem contaminar o domínio. Essas necessidades motivaram a evolução da proposta inicial para a plataforma PROCEL implementada.

## 1.2 Problema e objetivos

O problema investigado é: *como projetar e implementar uma arquitetura que integre telemetria heterogênea e dados de participação dos usuários, preserve rastreabilidade e forneça dados confiáveis para análise e gamificação da eficiência energética?*

O objetivo geral é projetar, implementar e avaliar essa arquitetura de backend. Os objetivos específicos são: identificar requisitos; definir persistência para eventos brutos e dados canônicos; implementar REST e MQTT com idempotência e reprocessamento; transformar formatos externos; oferecer API e console; integrar medições, participação, missões e XP sem duplicação; e avaliar o artefato por testes e experimento de desempenho. A futura aplicação de gamificação é consumidora prevista da API, não um produto desenvolvido neste trabalho.

## 1.3 Contribuições e organização

As contribuições são uma arquitetura que separa bruto, canônico, análise e gamificação; idempotência em múltiplas fronteiras; catálogo configurável de integrações; processamento recuperável por *claim*, *lease* e repetição; e evidências funcionais e de desempenho. Os capítulos seguintes apresentam fundamentação, trabalhos relacionados, método, implementação, avaliação e conclusão.

# 2 Fundamentação Teórica

## 2.1 IoT, heterogeneidade e canonicalização

IoT combina sensoriamento, comunicação e computação distribuída (Gubbi; Buyya; Marusic; Palaniswami, 2013). MQTT usa publicação–assinatura e níveis de qualidade de serviço. No QoS 1, a entrega é “pelo menos uma vez”, de modo que duplicatas podem ocorrer; o `PUBACK` transfere ao receptor a responsabilidade pela mensagem, mas não comprova que canonicalização, agregação ou concessão de XP tenham terminado (OASIS, 2019). Por isso, a identidade idempotente da aplicação não pode depender apenas do identificador de pacote MQTT. Fontes também podem expressar a mesma grandeza com nomes e estruturas distintas. Um modelo canônico desacopla produtores dos consumidores, enquanto a retenção bruta temporária sustenta auditoria e reprocessamento dentro do prazo configurado (Santos, 2026a).

## 2.2 Persistência poliglota

Bancos relacionais oferecem esquema, integridade referencial e transações. Bancos de documentos acomodam estruturas variáveis. A literatura recomenda escolher mecanismos conforme acesso, consistência e evolução, em vez de assumir uma tecnologia universal (Stonebraker; Cetintemel, 2005; Grolinger; Higashino; Tiwari; Capretz, 2013; Gessert; Wingerath; Friedrich; Ritter, 2017). No PROCEL, MongoDB preserva o evento bruto e PostgreSQL mantém domínio, agregados e gamificação.

## 2.3 Fluxos, idempotência e janelas

Fluxos de sensores são ilimitados e podem chegar fora de ordem. O processamento deve explicitar tempo do evento, janelas e gatilhos (Akidau; Bradshaw; Chambers et al., 2015). Intervalos semiabertos evitam dupla contagem nas fronteiras. Idempotência permite repetir operações sem multiplicar efeitos. Workers reservam itens atomicamente, usam *lease* temporário e recuperam trabalho após falhas.

## 2.4 Gamificação e energia

Gamificação emprega elementos de jogos em contextos não lúdicos (Deterding; Dixon; Khaled; Nacke, 2011). Pontos e rankings não bastam: autonomia, competência e relacionamento influenciam a motivação (Ryan; Deci, 2000). Estudos empíricos são majoritariamente positivos, mas dependem do contexto (Hamari; Koivisto; Sarsa, 2014). Para software gamificado, recomenda-se analisar contexto e usuários, definir objetivos, projetar e avaliar iterativamente (Morschheuser; Hassan; Werder; Hamari, 2018). A plataforma oferece missões auditáveis, mas não presume que XP equivale a economia.

No contexto da plataforma, a pessoa também funciona como fonte de dados: sua participação na aplicação cliente poderá produzir registros de adesão a missões, atividades, feedback e progresso. Esses dados móveis complementam a observação dos sensores ambientais. O backend implementado modela identidades, ocorrências, atividades e lançamentos de XP necessários a essa integração, embora a interface móvel que originará parte das interações permaneça externa ao artefato avaliado.

## 2.5 Segurança e observabilidade

IoT amplia desafios de autenticação, autorização, integridade e privacidade (Roman; Zhou; Lopez, 2013). A plataforma usa JWT e papéis; o serviço de telemetria não acessa o PostgreSQL. Correlação, estados e métricas permitem observar pipelines assíncronos, sem registrar segredos, dados pessoais ou payloads integrais (Santos, 2026b,c).

# 3 Trabalhos Relacionados

Orland et al. (Orland; Ram; Lang; Houser; Kling; Coccia, 2014) conectaram cargas de tomada monitoradas a um jogo em escritório e relataram redução média de 13% em relação à linha de base dessas cargas. Morganti et al. (Morganti; Pallavicini; Cadel; Candelieri; Archetti; Mantovani, 2017) incluíram dez estudos em sua revisão e encontraram potencial, porém evidência ainda limitada. Oppong-Tawiah et al. (Oppong-tawiah; Webster; Staples; Cameron; Guinea; Hung, 2020) aplicaram cinco ciclos de projeto a um aplicativo que monitorava equipamentos. Iria et al. (Iria; Fonseca; Cassola; Barbosa; Soares; Coelho; Ozdemir, 2020) combinaram painéis, competição e mensagens; os resultados da demonstração indicaram economia de 20%. Beck et al. (Beck; Chitalia; Rai, 2019), analisando 57 aplicativos, observaram subutilização de teoria comportamental. Esses resultados justificam investigar a infraestrutura necessária à intervenção, mas não são transferidos como estimativa de economia para o PROCEL.

A Tabela 1 separa mecanismo descrito, mecanismo avaliado e aspecto que a fonte não permite determinar. Essa distinção evita interpretar ausência de descrição como ausência de implementação. A lacuna explorada pelo TCC está na combinação, em um artefato, de retenção bruta delimitada, canonicalização configurável, agregação recuperável e efeitos idempotentes entre evidência, interpretação, evento e recompensa. A contribuição é de engenharia de dados; a efetividade comportamental permanece uma pergunta experimental posterior.

| **Trabalho** | **Objeto descrito** | **Mecanismos pertinentes** | **Limite da comparação** |
|---|---|---|---|
| Orland et al. | Intervenção com jogo e cargas monitoradas | Medição, linha de base e retorno ao usuário | Avalia comportamento; não detalha uma arquitetura reproduzível de ingestão. |
| Oppong-Tawiah et al. | Aplicativo sustentável desenvolvido em cinco ciclos | DSR, monitoramento de equipamentos e interação | Não permite determinar idempotência ou reprocessamento. |
| Iria et al. | Plataforma de gamificação para escritório | Medição real, painéis, competição e mensagens | Proveniência e consistência distribuída não são o objeto da avaliação. |
| Grolinger et al. | Levantamento de bancos NoSQL e NewSQL | Seleção por modelo, escalabilidade e consistência | Orienta a persistência, mas não propõe o pipeline PROCEL. |
| Akidau et al. | Modelo de processamento de fluxos não limitados | Tempo do evento, janelas, gatilhos e atraso | Fundamenta a semântica temporal; não trata gamificação. |

Tabela 1 – Síntese comportamental e arquitetural dos trabalhos relacionados

# 4 Metodologia

O trabalho adota Ciência do Projeto, na qual conhecimento é produzido pela construção e avaliação de artefatos relevantes (Hevner; March; Park; Ram, 2004; Gregor; Hevner, 2013). O processo foi organizado pelas seis atividades de Peffers et al. (Peffers; Tuunanen; Rothenberger; Chatterjee, 2007). A Tabela 2 explicita como essas atividades ocorreram, evitando apresentar o método apenas como uma sequência abstrata.

| **Atividade DSR** | **Execução no projeto** | **Evidência produzida** |
|---|---|---|
| Identificação do problema | Análise do objetivo institucional e dos riscos de integrar sensores heterogêneos a missões: duplicação, perda da origem, evolução de formato e efeitos repetidos. | Questão de pesquisa e requisitos arquiteturais. |
| Definição dos objetivos | Conversão dos riscos em propriedades observáveis: preservação delimitada, canonicalização, idempotência, recuperabilidade, rastreabilidade e autorização. | Objetivos da Seção 1.2 e critérios da avaliação. |
| Projeto e desenvolvimento | Construção incremental dos módulos Telemetry, API e Admin, das migrations e dos contratos REST/MQTT. | Código, esquema, documentação e diagramas. |
| Demonstração | Exercício dos fluxos de ingestão, consulta, missão e operação por testes automatizados e roteiro HTTP. | Classes de teste, relatório de testes de sanidade e exemplos de fluxo. |
| Avaliação | Inspeção de restrições e cenários automatizados; análise do relatório operacional; benchmark dos motores de regras. | Matriz propriedade–evidência e resultados do Capítulo 6. |
| Comunicação | Consolidação das decisões, limitações e instruções de reprodução. | Este TCC e documentos técnicos versionados. |

Tabela 2 – Correspondência entre a DSR e as atividades executadas

Os requisitos não foram obtidos por entrevistas ou por um protocolo formal com usuários finais. Eles foram reconstruídos a partir do objetivo do projeto institucional,

da evolução do artefato e de falhas plausíveis do pipeline. Consequentemente, a avaliação responde se a arquitetura implementa propriedades técnicas selecionadas; ela não valida usabilidade, motivação ou redução de consumo. Os ciclos foram incrementais: a separação entre bruto e canônico antecedeu a inclusão de perfis versionados; em seguida foram introduzidos processamento com *claim*/*lease*, agregação e, por fim, ocorrências, atividades e ledger de XP. O histórico Git e as migrations registram essa evolução, mas não foi mantido um diário experimental que permita atribuir cada mudança a uma sessão formal de avaliação.

A revisão de literatura foi narrativa, orientada por aderência ao problema, publicações revisadas por pares, DOI e fontes primárias para normas. O Qualis foi consultado apenas como informação auxiliar sobre veículos, pois sua unidade de avaliação, ciclo e critérios não substituem relevância temática nem constituem um protocolo reproduzível de busca (CAPES, 2023). Assim, este trabalho não reivindica revisão sistemática nem exaustividade.

## 4.1 Delimitação do artefato e protocolo de avaliação

A versão de referência do artefato técnico é o commit `141db8ccbf63cf3896344fbb90197dfae3618691`, de 2 de novembro de 2026, do repositório `PROCEL-Back-End`. Evidências posteriores ou oriundas de alterações locais não integram a avaliação. Para cada propriedade, foram procurados quatro tipos de evidência: restrição de armazenamento, cenário automatizado com oráculo explícito, relatório de execução e observação operacional. A presença de código ou de uma anotação `@Test`, isoladamente, não foi tratada como resultado.

Os módulos Java podem ser exercitados, a partir da raiz do repositório técnico, com `.\Procel-API\mvnw.cmd-fProcel-API/pom.xmltest` e `.\Procel-Telemetry\mvnw.cmd-fProcel-Telemetry/pom.xmltest`; os testes de integração usam Testcontainers para PostgreSQL, MongoDB e HiveMQ. O roteiro de testes de sanidade é executado separadamente contra um ambiente implantado. O benchmark usa o perfil `drools-benchmark`, com o comando registrado no documento de decisão técnica.

O equipamento informado como utilizado nas execuções locais foi inspecionado em 2 de novembro de 2026: Windows 11 Pro de 64 bits, versão 10.0.26300 (build 26300); processador Intel Core i5-11300H a 3,10 GHz, com quatro núcleos e oito processadores lógicos; e 16 GB de memória RAM. O ambiente do projeto aponta, por `JAVA_HOME`, para OpenJDK Temurin 21.0.10+7 de 64 bits; o Docker Engine identificado é a versão 28.5.2. Como o executável `java` disponível primeiro no `PATH` pertence ao Java 8, a reprodução deve conferir `JAVA_HOME` e a saída de `java -version` usada pelo Maven antes da execução. Versões de software observadas posteriormente podem diferir das presentes na data original de cada relatório.

A avaliação considerou correção funcional, integridade, interoperabilidade, recuperabilidade, segurança e desempenho. O relatório técnico do próprio projeto não constitui auditoria independente; o relatório de testes de sanidade antecede o commit de referência; e os testes automatizados não representam carga de produção, indisponibilidade prolongada ou todos os entrelaçamentos possíveis de concorrência.

# 5 Arquitetura e Desenvolvimento

## 5.1 Visão geral

O `Procel-Telemetry` recebe REST/MQTT e é proprietário do MongoDB; o `Procel-API` mantém autenticação, domínio, análise e gamificação no PostgreSQL; e o `Procel-Admin` oferece console React (Santos, 2026d).

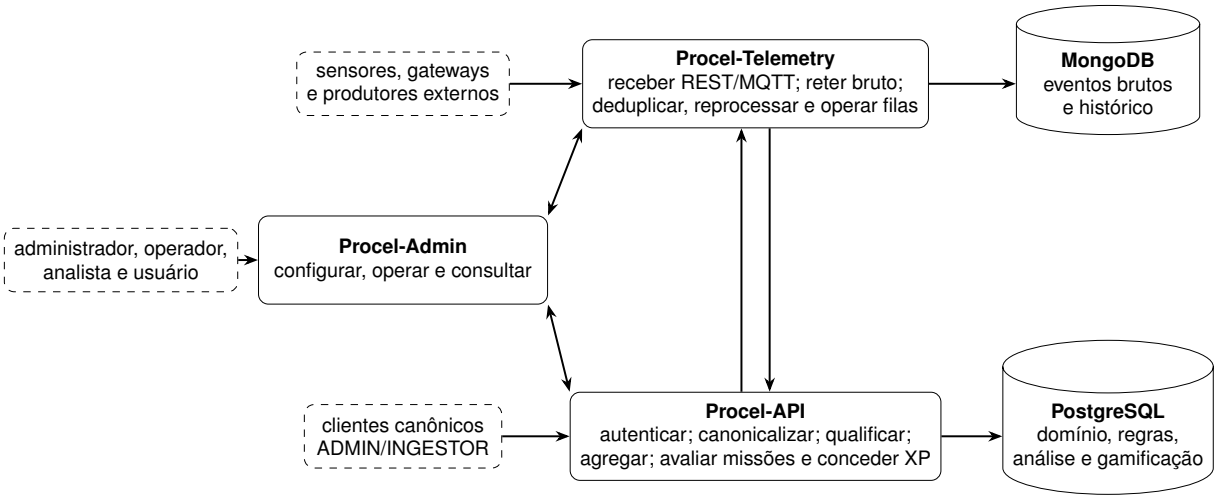



Figura 1 – Arquitetura do PROCEL, objetivos dos componentes e relações

A Figura 1 explicita quatro relações principais. Primeiro, produtores enviam mensagens ao `Procel-Telemetry`, cujo objetivo é aceitar heterogeneidade sem perder a evidência original; a gravação no MongoDB antecede o ACK sempre que o protocolo exige confirmação. Segundo, o worker recupera eventos, consulta na API o snapshot dos perfis e parsers e envia o resultado para a rota interna com JWT curto de `TELEMETRY_SERVICE`. Terceiro, o `Procel-API` transforma a entrada em medições canônicas, aplica regras, produz agregados e relaciona eventos de missão a atividades e XP, persistindo o estado transacional no PostgreSQL. Quarto, pessoas operam o sistema por meio do `Procel-Admin`, que consome as duas APIs com o JWT do usuário; clientes autorizados também podem enviar dados já canônicos diretamente à API. A futura aplicação móvel ocupará essa fronteira de cliente: consumirá missões e recompensas e enviará interações das pessoas como sensores, sem acesso direto aos bancos.

O limite de propriedade é deliberado: `Procel-Telemetry` não acessa o Post-

greSQL, e `Procel-API` não altera o documento bruto. MongoDB responde por preservação e reprocessamento; PostgreSQL, por integridade referencial e efeitos de domínio. Essa separação permite repetir a transformação sem reescrever a evidência e impede que uma retransmissão técnica gere uma segunda medição ou recompensa.

## 5.2 Console de administração

O `Procel-Admin` materializa a camada operacional da arquitetura. As Figuras 2–4 apresentam três exemplos do ambiente implantado: inspeção de medições e sua qualificação, configuração das regras aplicadas aos parâmetros dos sensores e definição de uma missão com recompensa. As capturas demonstram a correspondência entre a interface e os elementos do modelo, mas não substituem os testes funcionais discutidos no Capítulo 6 nem constituem uma avaliação de usabilidade.

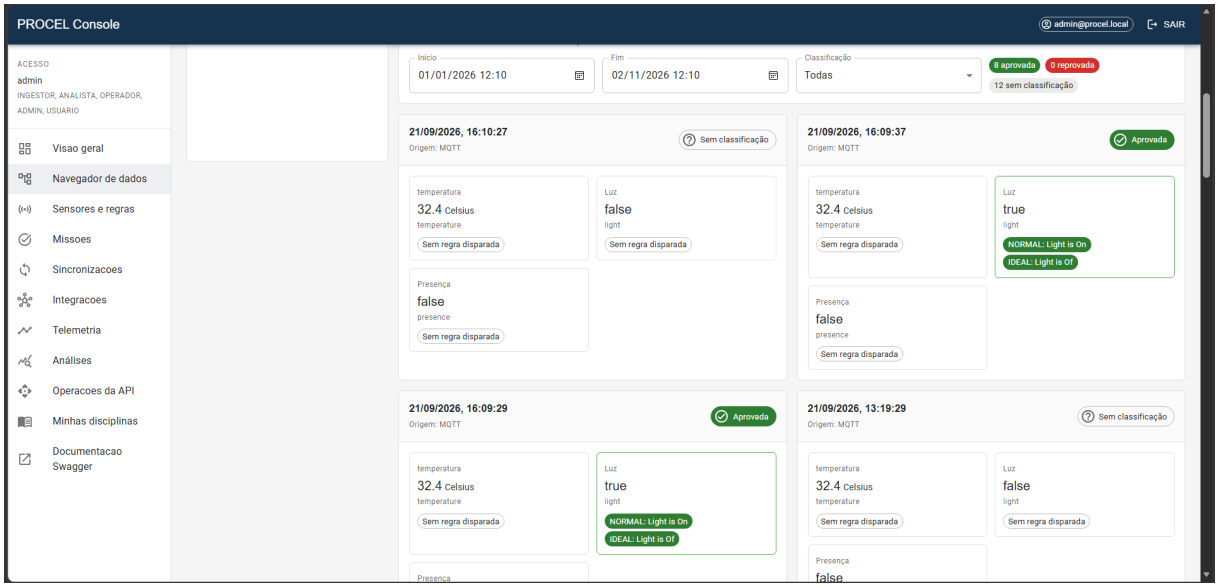


Figura 2 – Console de administração: medições recebidas por MQTT e resultados de qualificação

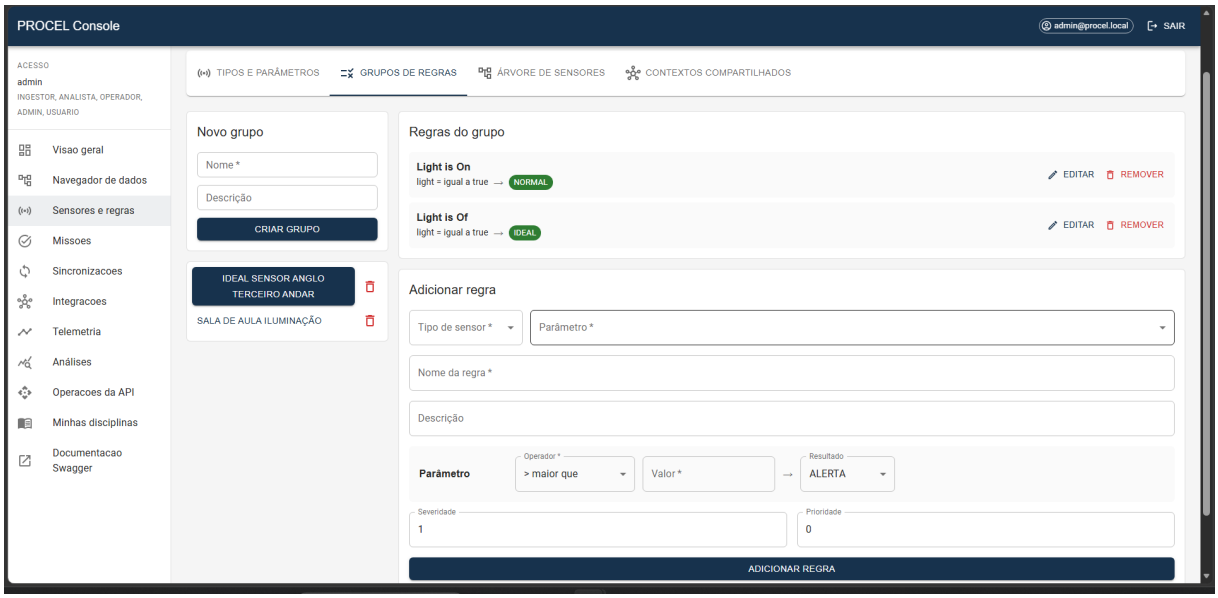



Figura 3 – Console de administração: definição de grupos e regras de qualificação

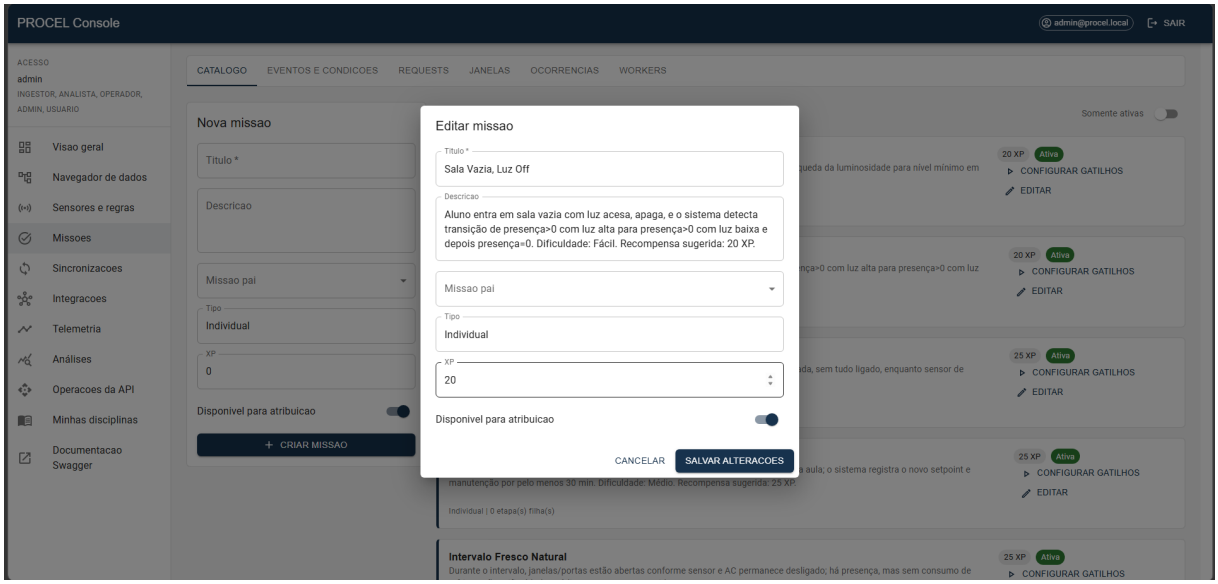



Figura 4 – Console de administração: edição de missão e recompensa em XP

## 5.3 Perfis de usuário e controle de acesso

O controle de acesso segue o modelo RBAC (*Role-Based Access Control*). A entidade `pessoa` concentra identidade, nome, e-mail, senha armazenada como *hash*, telefone, matrícula e vínculo opcional com curso. Os papéis não são gravados como uma única coluna: a relação `pessoa` 1:N `pessoa_role` permite que a mesma pessoa acumule responsabilidades, por exemplo `OPERADOR` e `ANALISTA`. Após a autenticação, esses papéis são incluídos no JWT e verificados nas rotas da API.

| **Papel** | **Natureza** | **Responsabilidades principais** |
|---|---|---|
| ADMIN | usuário humano | Administração ampla: pessoas e papéis, integrações, contextos compartilhados, catálogo, telemetria, missões, eventos, regras e jobs analíticos. Também consulta métricas operacionais. |
| OPERADOR | usuário humano | Operação cotidiana: cadastro e vínculo de sensores, sincronização de salas, grupos de regras, missões e criação ou acompanhamento de jobs, sem administrar credenciais e papéis globais. |
| ANALISTA | usuário humano | Consulta de medições, agregados, jobs, missões e informações operacionais necessárias à análise, sem alterar configurações sensíveis. |
| USUARIO | usuário humano | Funcionalidades de participante: autenticação, consulta do catálogo e de missões permitidas, atividades individuais, saldo e lançamentos de XP. |
| INGESTOR | identidade técnica | Envio de medições canônicas ou por perfil de integração. Seu escopo reduzido evita conceder privilégios administrativos a produtores de dados. |
| `TELEMETRY_SERVICE` | identidade técnica | Comunicação interna do `Procel-Telemetry` com o `Procel-API`, por JWT de curta duração, para canonicalizar eventos brutos previamente persistidos. |

Tabela 3 – Perfis e identidades técnicas do controle de acesso

A separação aplica o princípio do menor privilégio. O `USUARIO` participa da camada de gamificação, enquanto `ANALISTA` observa dados sem administrar o sistema; `OPERADOR` modifica configurações operacionais; e `ADMIN` mantém ações de maior impacto. As identidades técnicas não representam pessoas e não devem ser usadas para acesso interativo. Rotas públicas limitam-se à autenticação, cadastro habilitado, documentação e saúde; as demais combinam autenticação e autorização por papel. No domínio, uma pessoa pode relacionar-se a curso, disciplinas, presenças, atividades e lançamentos de XP, mas seu papel de segurança permanece independente desses

vínculos acadêmicos. Assim, ser aluno ou beneficiário de uma missão não concede automaticamente privilégio administrativo.

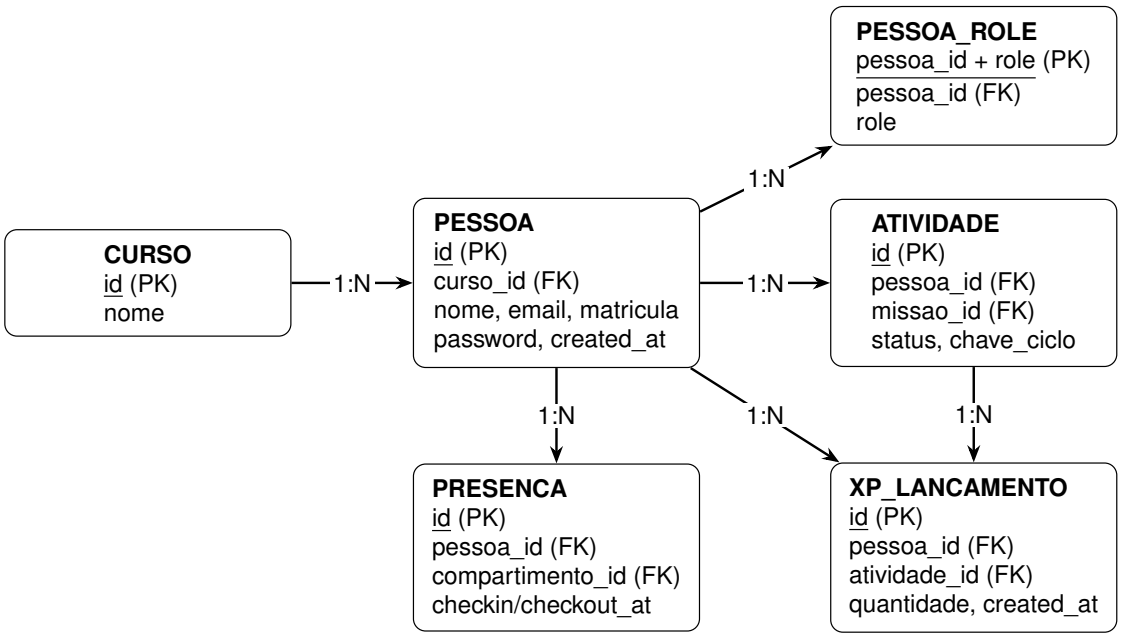



Figura 5 – Relações entre identidade, autorização e gamificação

A Figura 5 mostra por que papel, contexto e recompensa são conceitos distintos. `pessoa_role` autoriza operações; `curso` e `presenca` contextualizam a pessoa; `atividade` registra sua participação em uma missão; e `xp_lancamento` forma o histórico imutável da recompensa, ligado simultaneamente à pessoa e à atividade que o originou.

## 5.4 Modelagem relacional do domínio de sensores

O modelo relacional foi organizado para separar três conceitos que, em mensagens de dispositivos, frequentemente aparecem misturados: o *tipo* de equipamento, a *observação* realizada em um instante e a *interpretação* dessa observação. A Figura 6 apresenta esse núcleo. Ela foi reconstruída a partir das entidades JPA e das migrations Flyway atuais; campos secundários foram omitidos para preservar a legibilidade.

Os diagramas são deliberadamente divididos por domínio para manter as relações legíveis; eles não devem ser interpretados como inventários isolados de todo o banco. A inspeção das 32 migrations identificou 44 tabelas no PostgreSQL. Destas, 36 possuem classe JPA anotada com `@Entity`; as demais são tabelas associativas ou estruturas operadas diretamente por JDBC e coleções JPA. O MongoDB acrescenta a coleção documental `raw_telemetry_events`. A Tabela 4 resume o inventário; nomes, chaves, retenção e produtores de cada estrutura constam no *Catálogo de Dados* versionado (Santos, 2026f).

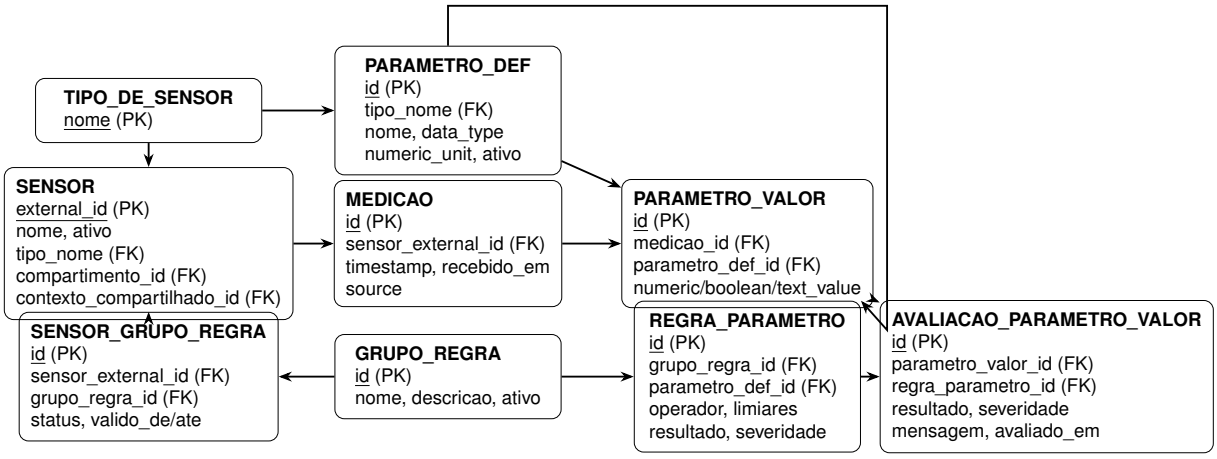



Figura 6 – DER do núcleo de sensores, regras e qualificação; todas as relações indicadas são 1:N

| Domínio | Estruturas | Objetivo |
|---|---|---|
| Espacial e acadêmico | 11 | Localizar sensores e relacionar pessoas, turmas, aulas e ocupação. |
| Sensores, valores e regras | 10 | Representar observações canônicas, idempotência e qualificação. |
| Perfis de integração | 4 | Converter formatos externos de maneira versionada e configurável. |
| Análise | 3 | Planejar janelas recuperáveis e armazenar agregados numéricos. |
| Missões e gamificação | 11 | Transformar evidências em ocorrências, progresso e recompensas auditáveis. |
| Contextos compartilhados | 5 | Reutilizar observações externas e controlar coleta, fonte e vigência. |
| Telemetria bruta (MongoDB) | 1 | Reter payload, identidade, hash, estados, tentativas e histórico de reprocessamento. |

Tabela 4 – Resumo das estruturas persistentes identificadas

### 5.4.1 Tipo, instância e localização do sensor

`tipo_de_sensor` funciona como um catálogo de capacidades. Uma instância física ou lógica é representada por `sensor`, cuja chave `external_id` corresponde ao identificador reconhecido fora do banco, como serial, MAC ou identificador do gateway. Assim, vários sensores do tipo `SII_SMART` compartilham definições de parâmetros, mas mantêm identidade e histórico independentes.

Cada sensor referencia exatamente um tipo. Quanto ao escopo espacial, a restrição `ck_sensor_exactly_one_scope` exige exatamente uma das alternativas: um `compartimento` interno ou um `contexto_compartilhado`, usado por observações externas, como condições meteorológicas comuns a várias salas. Essa exclusão evita associar simultaneamente o mesmo sensor a uma sala e a um contexto global.

### 5.4.2 Definição e valor de parâmetros

`parametro_def` descreve o contrato do dado: nome canônico, descrição, tipo `NUMERIC`, `BOOLEAN` ou `TEXT`, unidade e estado ativo. A unicidade de `tipo_nome + nome` impede duas definições com o mesmo significado dentro de um tipo de sensor. Uma definição possível é `temperatura`, numérica, medida em °C; outra é `presenca`, booleana e sem unidade.

`medicao` representa o envelope temporal produzido por um sensor. Uma medição pode conter vários `parametro_valor`; por exemplo, temperatura, umidade e presença observadas no mesmo instante. O par `medicao_id + parametro_def_id` é único, logo uma mesma medição não pode conter duas temperaturas canônicas.

O valor usa três colunas mutuamente interpretadas segundo `data_type`: `numeric_value`, `boolean_value` ou `text_value`. Esse modelo evita converter todos os valores para texto, preserva comparações numéricas e permite validar a compatibilidade entre definição, regra e valor.

| Definição | Tipo/unidade | Coluna usada | Exemplos |
|---|---|---|---|
| temperatura | NUMERIC / °C | numeric_value | 22,5; 29,1 |
| presenca | BOOLEAN | boolean_value | verdadeiro; falso |
| estado_porta | TEXT | text_value | ABERTA; FECHADA |

Tabela 5 – Exemplos de parâmetros tipados no modelo canônico

## 5.5 Modelagem das regras de qualificação

Uma regra não é ligada diretamente ao sensor. `grupo_regra` reúne uma política coerente, como “Conforto térmico de salas de aula”. A entidade associativa `sensor_grupo_regra` cria a relação muitos-para-muitos entre sensores e grupos, acrescentando estado e intervalo de validade. Essa modelagem permite ativar a mesma política em vários sensores, substituir uma política ao longo do tempo e manter vínculos em rascunho sem aplicá-los.

Cada `regra_parametro` pertence a um grupo e referencia exatamente uma `parametro_def`. Ela contém operador, limiar(es), resultado, severidade, prioridade e estado. Os operadores implementados são `GT`, `GTE`, `LT`, `LTE`, `EQ`, `NEQ`, `BETWEEN`, `OUTSIDE` e `CONTAINS`. A compatibilidade depende do tipo: números aceitam comparações e intervalos; booleanos, igualdade ou diferença; textos, igualdade, diferença ou contenção.

O processo de qualificação é mostrado na Figura 7. Para cada valor persistido, o serviço procura grupos do sensor com status `ATIVO` e vigentes no instante da medição. Em cada grupo, seleciona regras ativas da mesma definição de parâmetro. Toda regra satisfeita gera uma linha em `avaliacao_parametro_valor`.

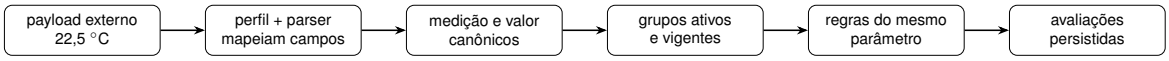


Figura 7 – Fluxo de canonicalização e qualificação de um valor

### 5.5.1 Exemplo completo de qualificação

Considere o sensor `SII-002`, do tipo `SII_SMART`, associado à sala 393. O tipo declara o parâmetro `temperatura` como numérico em °C. O sensor possui vínculo ativo com o grupo “Conforto térmico”, contendo as regras da Tabela 6.

| **Regra** | **Operador** | **Limiares** | **Resultado** | **Nível** |
|---|---|---|---|---|
| Faixa confortável | BETWEEN | 20,0 e 24,0 | IDEAL | 0 |
| Calor moderado | GT | 24,0 | ALERTA | 1 |
| Temperatura extrema | OUTSIDE | 16,0 e 28,0 | CRITICO | 3 |

Tabela 6 – Exemplo de regras para o parâmetro temperatura

Se o payload externo contiver `"temp": 22.5`, o parser converte `temp` para a definição canônica `temperatura`; cria uma medição e um `parametro_valor` com `numeric_value = 22.5`. Apenas `BETWEEN 20.0 24.0` é satisfeita, produzindo uma avaliação `IDEAL`. Para 29,1, as regras “Calor moderado” e “Temperatura extrema” são ambas verdadeiras, produzindo `ALERTA` e `CRITICO` separados.

Esse comportamento é deliberadamente não exclusivo: o avaliador percorre todas as regras e não interrompe no primeiro acerto. A restrição única `parametro_valor_id + regra_parametro_id` impede repetir a mesma avaliação, mas não impede resultados diferentes para o mesmo valor. A coluna `prioridade` existe no modelo, porém o serviço atual não a utiliza para escolher um único resultado. Caso a interface necessite de uma qualificação consolidada, deve adotar uma política explícita, por exemplo maior severidade, e preservá-la separadamente das avaliações elementares.

### 5.5.2 Relação entre qualificação e missões

Uma condição de evento pode operar de duas formas. Em `PARAMETRO_VALOR`, compara diretamente o dado canônico; em `AVALIACAO_REGRA`, referencia uma regra e o resultado esperado. Assim, a missão pode depender de uma política de domínio sem copiar seus limiares. Por exemplo, “Luz na Medida” pode exigir que luminosidade tenha avaliação `IDEAL` e presença seja verdadeira durante uma janela. A evidência da ocorrência mantém referência à avaliação que justificou a decisão, formando a cadeia mostrada na Figura 8.

Essa cadeia não depende apenas da proximidade conceitual das entidades. O documento MongoDB guarda `processing.canonicalMeasurementId`; no sentido inverso, `medicao_ingestao_metadata` registra `raw_telemetry_event_id`, produtor, mensagem, perfil e versão de parser. No PostgreSQL, chaves estrangeiras conectam medição a valores, evidências a ocorrências, ocorrências a atividades e lançamentos de XP. A

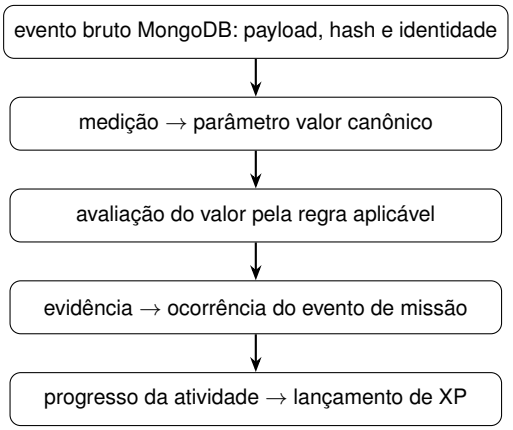


Figura 8 – Rastreabilidade do dado bruto ao efeito de gamificação

ligação MongoDB–PostgreSQL é lógica, não uma chave estrangeira distribuída; sua consistência depende da gravação dos identificadores pelos dois serviços.

Há ainda um limite histórico. Ocorrências de missão preservam um *snapshot* de contexto e referências às evidências, e versões de parser deixam de ser editáveis após ativação. As regras de qualificação, entretanto, podem ser alteradas sem que todos os seus limiares sejam copiados para a avaliação já persistida. Logo, a implementação permite explicar qual evidência e qual regra foram usadas, mas não garante reconstrução integral da política histórica em todos os casos. Versionar ou copiar os critérios de cada regra para a avaliação é trabalho futuro.

## 5.6 Telemetria bruta e MQTT

`raw_telemetry_events` registra produtor, origem, mensagem, sensor, instantes, payload, hash, estado e tentativas. A chave `producerId + source + messageId` implementa idempotência. Conteúdo divergente sob a mesma identidade gera conflito (Santos, 2026a).

O ciclo vai de `RECEIVED` a `PROCESSING` e então a aceito, duplicado, conflito, falho ou descartado. O worker usa *claim* e *lease*; falhas transitórias recebem repetição. O reprocessamento preserva payload, identidade e hash, mas seleciona a versão ativa do parser no novo processamento. Portanto, ele é uma *reinterpretação auditada*, não a reprodução garantida da transformação original. A entrada anterior, inclusive a versão de parser, permanece no histórico; se a nova interpretação colidir com a identidade canônica já persistida, a API deve responder como duplicata equivalente ou conflito, sem sobrescrita silenciosa.

Eventos brutos possuem TTL configurável: o padrão geral é 30 dias e o perfil de *staging* usa 7 dias, salvo configuração externa. Medições canônicas, metadados e buckets não têm expiração automática nesta etapa. Após o TTL, permanecem a medição e seu contexto de ingestão no PostgreSQL, mas o payload integral deixa de

estar disponível para nova interpretação. Assim, “preservação” neste trabalho significa preservação durante a janela de retenção, e não arquivo permanente.

No MQTT, o produtor vem do tópico. O ACK manual ocorre após persistência ou decisão permanente; falha transitória do MongoDB fica sem ACK para permitir nova entrega (Santos, 2026e; OASIS, 2019). Essa confirmação encerra a responsabilidade da etapa MQTT, não de todo o pipeline até o XP. Como o tópico participa da identidade, autenticidade depende de TLS, credenciais e ACL do broker que restrinja cada cliente ao próprio `producerId`. A aplicação documenta essa ACL, mas não consegue garanti-la quando o broker é implantado com configuração diferente (Santos, 2026b).

## 5.7 Domínio canônico e análise

No PostgreSQL, sensores, tipos, parâmetros, medições e valores formam o núcleo. Valores numéricos, booleanos e textuais são tipados. Metadados guardam contexto e idempotência. Perfis, versões de parser, mapeamentos e bindings transformam fontes sem ambiguidade (Santos, 2026f).

Jobs analíticos dividem períodos em janelas semiabertas $[t_i, t_f)$ e recuperáveis. Buckets armazenam média, mínimo, máximo e número de amostras por sensor, parâmetro, intervalo e versão. A média é aritmética por amostra; ao combinar buckets, ela é ponderada pela quantidade de amostras, não pela duração representada por cada leitura. Sensores com frequências diferentes, portanto, não devem ter suas amostras interpretadas como observações temporalmente equivalentes sem tratamento adicional. Mensagens atrasadas incluídas no intervalo podem ser incorporadas por recomputação explícita; não há *watermark* nem recálculo automático de bucket já encerrado. A API oferece consultas filtradas e paginadas; a cobertura visual do Admin não é usada como evidência da semântica de agregação.

## 5.8 Missões e XP

Eventos configuráveis referenciam parâmetros canônicos. Solicitações, ocorrências e evidências persistem a decisão. Janelas temporais usam intervalos semiabertos, lacuna máxima configurável e estados persistentes para duração, transição e encerramento. Ocorrências progridem atividades de modo idempotente. XP é um ledger append-only, cujo saldo é calculado por soma. O motor simples é padrão; Drools é opt-in, isolado da persistência e mantido experimental atrás de configuração (Santos, 2026g).

# 6 Avaliação e Discussão

## 6.1 Critérios e resultados funcionais

A avaliação foi organizada pelas propriedades derivadas do problema de pesquisa, não pelo tamanho do código. Contagens de 32 migrations, 30 controladores e 358 anotações JUnit descrevem a extensão do artefato, mas não são usadas como prova de correção ou cobertura. A Tabela 7 apresenta os cenários encontrados e o alcance efetivo da evidência.

| **Propriedade** | **Cenário e oráculo** | **Evidência observada** | **Conclusão delimitada** |
|---|---|---|---|
| Idempotência bruta e de XP | Ingestões concorrentes equivalentes devem criar um vencedor; conteúdo divergente deve conflitar; repetição de recompensa deve manter um lançamento. | `TelemetryMongoIntegrationTest` exercita concorrência e conflito; `XpRewardServiceIntegrationTest` exercita repetição, concorrência e conteúdo divergente. | Sustentada nos componentes testados; falta ensaio único de ponta a ponta até XP. |
| Recuperabilidade | Dois workers não devem reclamar o mesmo item; lease expirado deve permitir retomada; rollback não deve deixar janela parcial. | Testes de telemetria e `EventoJanelaAvaliacaoServiceTest` cobrem claim concorrente, lease, repetição, reinício simulado e rollback. | Sustentada no nível de serviço e banco; não equivale a teste de queda real dos processos. |
| Consistência entre serviços | Falha após o commit da API e antes da atualização do MongoDB deve resultar em repetição reconhecida, sem nova medição. | A fronteira transacional e as respostas `CREATED/DUPLICATE/CONFLICT` têm testes separados. Não foi localizado um teste que interrompa exatamente entre os dois commits. | Parcial; o cenário distribuído permanece experimento necessário. |
| Integração de formatos | Formatos externos diferentes para a mesma grandeza devem produzir valores canônicos equivalentes. | Há testes de seleção de perfil, mapeamento, tipo e versão de parser, mas não uma matriz publicada de fabricantes e payloads equivalentes. | Parcial; valida o mecanismo, não ampla interoperabilidade. |
| Janelas e agregação | Início deve ser incluído, fim excluído; repetição não deve duplicar buckets; atraso deve seguir política conhecida. | `NumericBucketAggregationTest` cobre intervalo semiaberto, precisão, recomputação, concorrência e rollback; janelas de missão cobrem lacuna e estados. | Sustentada para os casos testados; atraso exige recomputação explícita. |
| Autorização | Papéis permitidos devem obter sucesso e papéis proibidos, HTTP 403. | Testes de controladores exercitam ADMIN, ANALISTA, OPERADOR e USUARIO; o teste de sanidade confirma acesso do usuário às próprias atividades e rejeição de atividade alheia. | Sustentada nas rotas exercitadas, não como auditoria completa de segurança. |

Tabela 7 – Matriz de propriedades, cenários e alcance das evidências

Os relatórios Surefire correspondentes aos cinco conjuntos destacados registram 40 casos sem falhas: 8 de telemetria bruta, 10 de XP, 2 de fronteira transacional da ingestão, 9 de agregação e 11 de janelas de missão. Eles foram gerados em execuções distintas entre setembro e novembro de 2026, e não em uma única execução limpa e

arquivada do commit de referência. Por isso, sustentam os cenários específicos, mas não autorizam afirmar que toda a suíte da versão foi aprovada.

## 6.2 Relatório de testes de sanidade

O relatório de 17 de setembro de 2026 registra 112 verificações aprovadas, cinco avisos e zero falhas em 71,6 segundos contra o ambiente identificado como produção (Santos, 2026h). Os avisos foram: timeout de 45,2 segundos na sincronização de salas; HTTP 401 ao consultar o status do job de horários; verificação de persistência desse job ignorada em consequência; HTTP 409 ao tentar recriar um usuário já existente; e HTTP 404 na limpeza de uma presença que já não estava aberta. Os dois últimos eram estados tolerados pelo roteiro; os três primeiros revelam uma lacuna real na sincronização assíncrona e na renovação ou propagação do token.

Esse relatório antecede o commit avaliado e não contém o hash da versão implantada. Ele demonstra que um conjunto de rotas respondeu conforme o roteiro naquele ambiente e instante, mas não valida automaticamente a versão de novembro. O tempo total também não é medida de desempenho: mais de 60% dele corresponde ao timeout de uma única operação, e as etapas misturam rede, autenticação, persistência e verificações locais.

## 6.3 Experimento dos motores de regras

O benchmark JMH foi executado no equipamento descrito na Seção 4.1, com Windows 11, OpenJDK 21.0.10, processador Intel Core i5-11300H de quatro núcleos e oito processadores lógicos e 16 GB de RAM. A configuração empregou dois forks, cinco iterações de aquecimento, dez de medição, um segundo por iteração, uma thread nos resultados principais e o *profiler* de coleta de lixo. Foram usados os modos `SampleTime` e `Throughput`; os percentis vieram das amostras do primeiro. A persistência, a rede e os workers não participam da medição (Santos, 2026g).

Os cenários instantâneos usam a mesma definição simples e o mesmo conjunto de fatos. “Drools frio” cria um novo motor por operação e inclui geração/compilação da regra, construção da `KieBase` e avaliação; “Drools quente” reutiliza a base em cache, mas cria uma `KieSession` por avaliação. Cada operação avalia um lote completo, não um único fato. Com 1000 fatos, o motor simples teve média de 0,016 ms e p99 de 0,049 ms; Drools frio, 44,619 ms e 97,750 ms; Drools quente, 1,502 ms e 3,011 ms. O resultado sustenta apenas que o motor simples apresentou menor latência para as regras instantâneas medidas.

No cenário de duração, Drools processou 5000 fatos com média de 6,907 ms e p99 de 11,026 ms. Não existe resultado equivalente do motor simples para esse cenário;

portanto, esse número demonstra viabilidade local, não superioridade. A decisão de manter Drools opcional decorre de sua expressividade temporal, combinada com limites, cache e isolamento, e permanece condicionada a benchmark em hardware representativo e a um SLO definido.

## 6.4 Discussão e ameaças à validade

A combinação entre documento bruto, metadados canônicos, evidências, atividades e ledger torna a cadeia tecnicamente inspecionável durante a retenção do bruto. Restrições únicas e transações locais reduzem duplicação, enquanto estados persistidos e leases permitem retomada. Essas observações apoiam a viabilidade da arquitetura, mas não provam disponibilidade, correção universal ou adequação sob carga real.

As principais ameaças à validade interna são relatórios produzidos pela própria equipe, execuções não consolidadas em um pacote imutável e ausência do cenário exato de falha entre os dois bancos. O benchmark foi executado em um único computador de uso geral; processos concorrentes, políticas de energia e variações térmicas não foram registrados, o que limita sua comparação externa. Para validade externa, foram avaliados poucos formatos de dispositivos, um único contexto institucional e nenhuma carga representativa de uma edificação ocupada. Para validade de construto, sucesso técnico, XP e economia de energia são conceitos distintos: o estudo mede propriedades do software, não mudança comportamental nem consumo evitado. Permanecem pendentes backup e restauração ensaiados, rate limiting, carga contínua, caos, auditoria independente de segurança e estudo com usuários.

# 7 Conclusão

Este trabalho materializou a proposta inicial em uma arquitetura com três aplicações e dois modelos de persistência. O armazenamento bruto retém eventos REST/MQTT por prazo configurável; o modelo canônico estabiliza sensores e medições; perfis desacoplam produtores; janelas apoiam análise recuperável; e missões relacionam evidências, atividades e XP com mecanismos de idempotência. Assim, o backend integra estruturas documentais e relacionais, fontes ambientais heterogêneas e os registros de participação que a futura aplicação móvel produzirá com pessoas atuando como sensores.

As evidências sustentam propriedades específicas nos cenários automatizados inspecionados e justificam o motor simples como padrão nas regras instantâneas medidas. Elas não constituem prova de ausência de defeitos, consistência sob toda falha distribuída ou desempenho de produção. A contribuição central é integrar evidência bruta, interpretação canônica e efeitos de gamificação rastreáveis; não afirmar economia já alcançada.

Trabalhos futuros incluem executar e arquivar a matriz completa em um commit limpo; interromper o pipeline entre os commits MongoDB/PostgreSQL; versionar integralmente políticas históricas; validar novos fabricantes; testar carga, caos, backup e restauração; definir SLOs e alertas; repetir o benchmark em infraestrutura representativa; desenvolver e integrar a aplicação móvel prevista; e realizar estudo de campo com linha de base e grupo de comparação.

# 8 Artefatos técnicos consultados

Foram consultados, no commit `141db8ccbf63cf3896344fbb90197dfae3618691`, o README do projeto e os documentos `Catalogo-Dados.md`, `Modelo-Telemetria-MongoDB.md`, `MQTT.md`, `Seguranca-e-Operacao.md`, `Observabilidade.md`, `Decisao-Drools-POC.md` e `ApiSmokeReport-latest.md`, além de código, migrations, contratos HTTP, classes de teste e relatórios Surefire locais. A autoria “Santos (2026a–h)” nas referências designa o autor deste TCC como responsável pela documentação técnica do artefato; os títulos e o hash distinguem cada item. Para preservação de longo prazo, a versão entregue deve receber uma tag ou arquivo permanente contendo código, relatórios e resultados brutos do benchmark.

# Referências


AKIDAU, T.; BRADSHAW, R.; CHAMBERS, C. et al. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. **Proceedings of the VLDB Endowment**, [S.l.], v.8, n.12, p.1792–1803, 2015.

AL-FUQAHA, A.; GUIZANI, M.; MOHAMMADI, M.; ALEDHARI, M.; AYYASH, M. Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. **IEEE Communications Surveys & Tutorials**, [S.l.], v.17, n.4, p.2347–2376, 2015.

BECK, A. L.; CHITALIA, S.; RAI, V. Not so Gameful: A Critical Review of Gamification in Mobile Energy Applications. **Energy Research & Social Science**, [S.l.], v.51, p.32–39, 2019.

CAPES. **Metodologia do Qualis Referência**: Quadriênio 2017–2020. 2023. Acesso em: 2 nov. 2026, `https://www.gov.br/capes/pt-br/acesso-a-informacao/acoes-e-programas/avaliacao/avaliacao-quadrienal/avaliacao-quadrienal-2017-2020/metodologia-do-qualis-referencia-quadrienio-2017-2020`.

DETERDING, S.; DIXON, D.; KHALED, R.; NACKE, L. From Game Design Elements to Gamefulness: Defining Gamification. In: INTERNATIONAL ACADEMIC MINDTREK CONFERENCE, 15., 2011. **Proceedings...** [S.l.: s.n.], 2011. p.9–15.

GESSERT, F.; WINGERATH, W.; FRIEDRICH, S.; RITTER, N. NoSQL Database Systems: A Survey and Decision Guidance. **Computer Science – Research and Development**, [S.l.], v.32, p.353–365, 2017.

GREGOR, S.; HEVNER, A. R. Positioning and Presenting Design Science Research for Maximum Impact. **MIS Quarterly**, [S.l.], v.37, n.2, p.337–355, 2013.

GROLINGER, K.; HIGASHINO, W. A.; TIWARI, A.; CAPRETZ, M. A. M. Data Management in Cloud Environments: NoSQL and NewSQL Data Stores. **Journal of Cloud Computing**, [S.l.], v.2, 2013.

GUBBI, J.; BUYYA, R.; MARUSIC, S.; PALANISWAMI, M. Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions. **Future Generation Computer Systems**, [S.l.], v.29, n.7, p.1645–1660, 2013.

HAMARI, J.; KOIVISTO, J.; SARSA, H. Does Gamification Work? A Literature Review of Empirical Studies on Gamification. In: HAWAII INTERNATIONAL CONFERENCE ON SYSTEM SCIENCES, 47., 2014. **Anais...** [S.l.: s.n.], 2014. p.3025–3034.

HEVNER, A. R.; MARCH, S. T.; PARK, J.; RAM, S. Design Science in Information Systems Research. **MIS Quarterly**, [S.l.], v.28, n.1, p.75–105, 2004.

IRIA, J.; FONSECA, N.; CASSOLA, F.; BARBOSA, A.; SOARES, F.; COELHO, A.; OZDEMIR, A. A Gamification Platform to Foster Energy Efficiency in Office Buildings. **Energy and Buildings**, [S.l.], v.222, p.110101, 2020.

MORGANTI, L.; PALLAVICINI, F.; CADEL, E.; CANDELIERI, A.; ARCHETTI, F.; MANTOVANI, F. Gaming for Earth: Serious Games and Gamification to Engage Consumers in Pro-Environmental Behaviours for Energy Efficiency. **Energy Research & Social Science**, [S.l.], v.29, p.95–102, 2017.

MORSCHHEUSER, B.; HASSAN, L.; WERDER, K.; HAMARI, J. How to Design Gamification? A Method for Engineering Gamified Software. **Information and Software Technology**, [S.l.], v.95, p.219–237, 2018.

OASIS. **MQTT Version 5.0**. 2019. Editores: Andrew Banks, Ed Briggs, Ken Borgendale e Rahul Gupta. Acesso em: 2 nov. 2026, OASIS Standard. Disponível em: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>.

OPPONG-TAWIAH, D.; WEBSTER, J.; STAPLES, D. S.; CAMERON, A.-F.; GUINEA, A. Ortiz de; HUNG, T. Y. Developing a Gamified Mobile Application to Encourage Sustainable Energy Use in the Office. **Journal of Business Research**, [S.l.], v.106, p.388–405, 2020.

ORLAND, B.; RAM, N.; LANG, D.; HOUSER, K.; KLING, N.; COCCIA, M. Saving Energy in an Office Environment: A Serious Game Intervention. **Energy and Buildings**, [S.l.], v.74, p.43–52, 2014.

PEFFERS, K.; TUUNANEN, T.; ROTHENBERGER, M. A.; CHATTERJEE, S. A Design Science Research Methodology for Information Systems Research. **Journal of Management Information Systems**, [S.l.], v.24, n.3, p.45–77, 2007.

ROMAN, R.; ZHOU, J.; LOPEZ, J. On the Features and Challenges of Security and Privacy in Distributed Internet of Things. **Computer Networks**, [S.l.], v.57, n.10, p.2266–2279, 2013.

RYAN, R. M.; DECI, E. L. Intrinsic and Extrinsic Motivations: Classic Definitions and New Directions. **Contemporary Educational Psychology**, [S.l.], v.25, n.1, p.54–67, 2000.

SAGL, G.; RESCH, B.; BLASCHKE, T. Contextual Sensing: Integrating Contextual Information with Human and Technical Geo-Sensor Information for Smart Cities. **Sensors**, [S.l.], v.15, n.7, p.17013–17035, 2015.

SANTOS, R. A. d. **Modelo Documental da Telemetria no MongoDB**. 2026. Caminho: Documentos/Modelo-MongoDB/Modelo-Telemetria-MongoDB.md; acesso em: 2 nov. 2026, Documento técnico do repositório PROCEL-Back-End. Disponível em: <https://github.com/ravilon/PROCEL-Back-End/tree/141db8c>.

SANTOS, R. A. d. **Segurança e Operação**. 2026. Caminho: Documentos/Seguranca-e-Operacao.md; acesso em: 2 nov. 2026, Documento técnico do repositório PROCEL-Back-End. Disponível em: <https://github.com/ravilon/PROCEL-Back-End/tree/141db8c>.

SANTOS, R. A. d. **Observabilidade**. 2026. Caminho: Documentos/Observabilidade.md; acesso em: 2 nov. 2026, Documento técnico do repositório PROCEL-Back-End. Disponível em: <https://github.com/ravilon/PROCEL-Back-End/tree/141db8c>.

SANTOS, R. A. d. **PROCEL**: Arquitetura, Fluxos e Operação. 2026. Commit 141db8ccbf63cf3896344fbb90197dfae3618691; acesso em: 2 nov. 2026, README do repositório PROCEL-Back-End. Disponível em: <https://github.com/ravilon/PROCEL-Back-End/tree/141db8c>.

SANTOS, R. A. d. **MQTT no Procel-Telemetry**. 2026. Caminho: Documentos/MQTT.md; acesso em: 2 nov. 2026, Documento técnico do repositório PROCEL-Back-End. Disponível em: <https://github.com/ravilon/PROCEL-Back-End/tree/141db8c>.

SANTOS, R. A. d. **Catálogo de Dados**. 2026. Caminho: Documentos/Catalogo-Dados.md; acesso em: 2 nov. 2026, Documento técnico do repositório PROCEL-Back-End. Disponível em: <https://github.com/ravilon/PROCEL-Back-End/tree/141db8c>.

SANTOS, R. A. d. **Decisão Técnica**: Drools na Avaliação de Missões. 2026. Caminho: Documentos/Decisao-Drools-POC.md; acesso em: 2 nov. 2026, Documento técnico e benchmark do repositório PROCEL-Back-End. Disponível em: <https://github.com/ravilon/PROCEL-Back-End/tree/141db8c>.

SANTOS, R. A. d. **PROCEL API Smoke Test Report**. 2026. Caminho: Documentos/ApiSmokeTests/ApiSmokeReport-latest.md; 112 verificações aprovadas, 5 avisos e 0 falhas; execução em 17 set. 2026; acesso em: 2 nov. 2026, Relatório técnico do repositório PROCEL-Back-End. Disponível em: <https://github.com/ravilon/PROCEL-Back-End/tree/141db8c>.

STONEBRAKER, M.; CETINTEMEL, U. One Size Fits All: An Idea Whose Time Has Come and Gone. In: INTERNATIONAL CONFERENCE ON DATA ENGINEERING, 21., 2005. **Anais...** [S.l.: s.n.], 2005. p.2–11.